มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK

BUSINESS INTELLIGENCE

Chapter 19

Data Modeling & Data Analysis Expressions (DAX)

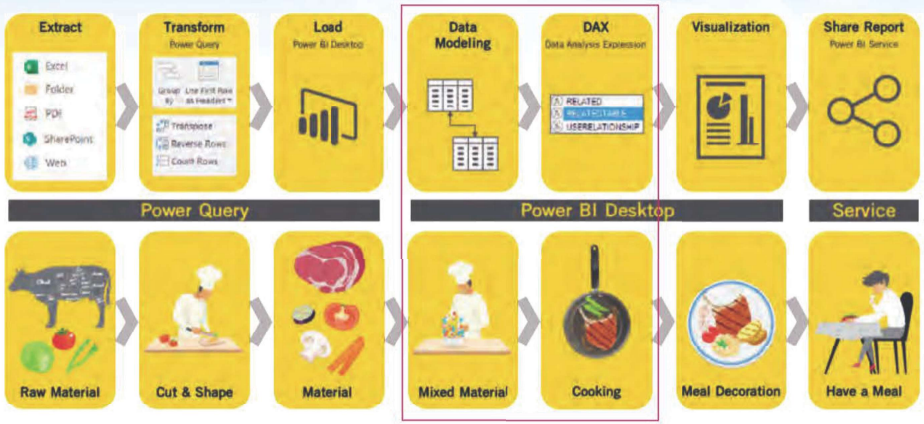
Asst.Prof. Sakchai Tangprasert (Ph.D. in IT)
Sakchai.t@sci.kmutnb.ac.th

Analytics and Big Data for Accountants
By Jim Lindell
© 2017 Association of International Certified Professional Accountants, Inc.

1. Data Modeling & DAX

- หลังจากเตรียมข้อมูลด้วยการ Clean & Transform Data จนได้ข้อมูลเฉพาะกิจที่จะนำมาใช้งานแล้ว เปรียบได้กับการหั่นเนื้อหั่นผักไว้ให้พร้อมเพื่อนำไปปรุงเป็นอาหาร แต่ก่อนที่จะนำสิ่งที่เตรียมไว้ไปใช้ ยังมีอีกขั้นตอนหนึ่งคือ "สูตรหรือวิธีการปรุงอาหาร" ตามเมนูให้อร่อย เปรียบขั้นตอนนี้เหมือนกับการทำ Data Modeling (คลุกเคล้าเครื่องปรุง) และ Data Analysis Expressions (DAX) (สูตรปรุงสุก) ก็จะได้ข้อมลสำคัญที่นำไป Visualize ได้ทันที

2. Mixed Material and Cooking



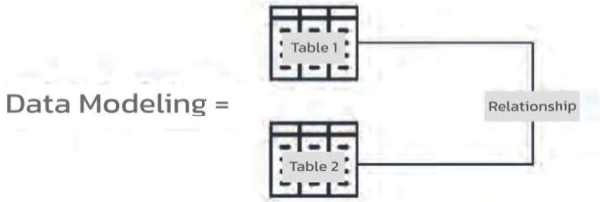
3. Data-Driven Business/Organization

- การสร้าง Data Model และการเขียนสูตร DAX (Data Analysis Expressions) ซึ่งเป็นส่วนที่ต้องทำก่อนจะถึงขั้นตอนของการออกแบบสร้างสรรค์ชาร์ตในรูปแบบต่าง ๆ (Visualization) ที่จะช่วยอธิบายข้อมูลให้เห็นได้ด้วยตา และทำให้ทุกคนเห็นภาพอาจเป็นหลักฐานเดียว แต่เป็นข้อพิสูจน์ชิ้นสำคัญที่จะนำไปสู่การหาข้อสรุป เพื่อดำเนินการอย่างหนึ่งต่อไป ภายใต้แนวคิดใหม่ในเวลานี้คือ Data-Driven Business/Organization

4. Data Modeling หรือ Model Relationships

- ก่อนปรุงอาหารนั้นจะต้องมีการตระเตรียมวัตถุดิบและเครื่องปรุงต่างๆ ที่จะนำมาใช้ในการประกอบอาหาร ซึ่งในส่วนนี้เรียกว่า "Data Modeling" โดยมีองค์ประกอบ 2 ส่วนที่สำคัญ นั่นก็คือ Table (ตารางข้อมูล) และ Relationship (ความสัมพันธ์)

Data Modeling = Tables+Relationships



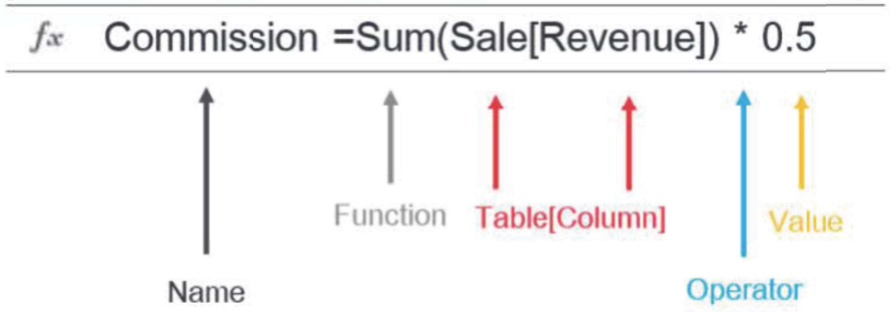
4. Data Modeling หรือ Model Relationships (ต่อ)

- จากรูปจะเห็นว่า Data Modeling นั้นจะมี Table มากกว่า 1 Table ขึ้นไป แล้วตัว Relationship จะมีหน้าที่ในการ Filter ข้อมูลที่มีความสัมพันธ์กันให้แสดงข้อมูลเชื่อมโยงซึ่งกันและกันได้ ฉะนั้นความสำคัญของ Data Modeling ก็คือ ถ้ามีการสืบค้นข้อมูลจากแหล่งต่าง ๆ มาวิเคราะห์ร่วมกัน ก็จำเป็นจะต้องทำ Data Modeling ก่อน

5. DAX Syntax

- หลังจากเตรียมเครื่องปรุงไว้พร้อมแล้ว (Data Relationship) ขั้นตอนนี้เปรียบได้กับการปรุงอาหาร ซึ่งอาจหมายถึง การปรุงสุก เช่น การอบ นึ่ง ทอด ผัด โดย DAX นั่นก็คือ "ภาษาในการเขียนสูตร" หรือในภาษาวิชาการอาจเรียกว่า "นิพจน์วิเคราะห์ข้อมูล" อันประกอบด้วยชุดของฟังก์ชัน (Functions), ตัวดำเนินการ (Operations) และค่าคงที่ (Constants) คล้าย ๆ กับที่นิยมใช้กับ Power Pivot ใน Excel

6. โครงสร้าง DAX Syntax



6. โครงสร้าง DAX Syntax (ต่อ)

- จากตัวอย่าง จะเป็นการหาค่าคอมมิชชัน (Commission Fee) ให้กับพนักงานขาย
  - Commission : ชื่อของสูตรในตัวอย่างนี้เท่านั้น (เปลี่ยนชื่อได้)
  - Sum : ชื่อฟังก์ชันที่ใช้ในการหาผลบวกของข้อมูลตัวเลข
  - Sale : ชื่อตารางที่จะใช้ในสูตรคำนวณ
  - Revenue : ชื่อของ Column ที่จะเลือกเพื่อใช้ในสูตรคำนวณ
  - * : เครื่องหมายดอกจันใช้แทนการคูณในสูตรคำนวณ
  - 0.5 : ตัวเลขที่จะนำมาใช้ในสูตรคำนวณ

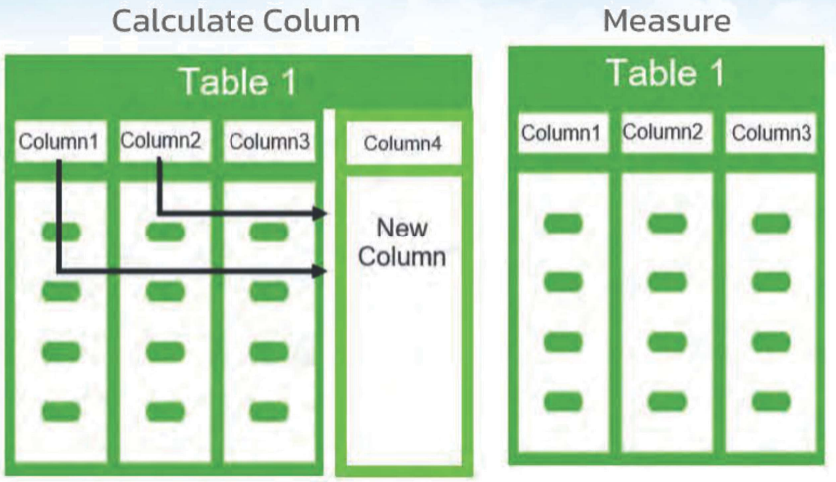
7. Calculate Column

- Calculate Column จะเป็นการสร้างสูตรเพิ่มข้อมูลเข้าไปใน Table เช่น Column1 มีข้อมูล 1 ล้านแถว นั่นแสดงว่า ถ้าหากเราสร้างสูตร Calculate Column ที่ Column 4 ขึ้นมา แสดงว่าเราจะมีข้อมูลเพิ่มอีก 1 ล้านแถวใน Column 4 ซึ่งการที่ข้อมูลเพิ่มขึ้นก็จะทำให้ข้อมูลมีขนาดใหญ่ขึ้นด้วย อาจส่งผลต่อประสิทธิภาพในการคำนวณข้อมูล ซึ่งในการสร้างสูตรโดยใช้ Calculate Column นั้นจะเรียกว่า Row Context จะเป็นการคำนวณทีละแถว ๆ ซ้ำไปเรื่อยจะครบทุกแถว เช่น สูตรต้องการบวกตัวเลขใน Column1 และ Column2 แล้วแสดงผลลัพธ์ที่ Column 4 โดยใช้สูตร Calculate Column (Row Context) ที่จะคำนวณไปทีละแถว ถ้าสมมุติว่ามีทั้งหมด 1 ล้านแถว ก็จะต้องคำนวณถึง 1 ล้านครั้ง ซึ่งจะส่งผลต่อความเร็วในการคำนวณอย่างแน่นอน

8. Measure

- Measure จะเป็นการสร้างสูตรที่ไม่มีการเพิ่มข้อมูลเข้าไปในตาราง คือพอเขียนสูตรแล้วจะยังไม่เห็นเหมือนกับวิธี Calculate Column แต่เมื่อไรที่ต้องการดูผลลัพธ์ก็จะต้องนำสูตรที่เขียนไปแสดงผลในมุมมอง Report จะเห็นว่า Measure ไม่ได้มีการเพิ่มขนาดของข้อมูล และก็ยังไม่ได้คำนวณทันทีจนกว่าจะเรียกใช้ในมุมมอง Report จะคำนวณก็ต่อเมื่อมีการสร้าง Visualize ทั้งนี้เป็นการลดภาระเครื่องโดยไม่จำเป็น เพราะถ้าต้องคำนวณแล้วนำผลลัพธ์ไปใส่คอลัมน์ทุกแถวของคอลัมน์แบบที่ใช้ใน Calculate Column ก็จะเป็นภาระมาก เพราะฉะนั้นถ้าเป็นไปได้ควรเลือกใช้วิธี Measure มากกว่าวิธี Calculate Column

9. ความแตกต่างของสูตร Calculate Column และ Measure



10. ความแตกต่าง Calculate Column และ Measure (ต่อ)

Calcuated Column	Measure
เก็บไว้ที่ Memory (RAM)	ไม่ได้เก็บ
สูตรจะคำนวณอีกครั้งก็ต่อเมื่อทำการ Refresh ข้อมูล	สูตรคำนวณตลอดเมื่ออยู่บน Report
ใช้ RAM ในการประมวลผล	ใช้ CPU ในการประมวลผล
ข้อมูลจะสามารถเห็นได้ใน Data View หรือ Table	ข้อมูลจะเห็นได้ก็ต่อเมื่อแสดงใน Report
การ Calculate Column ถ้าเป็นไปได้ควรทำใน Power Query ให้เสร็จเรียบร้อย	การใช้ DAX ใน Measure สำหรับการคำนวณ เป็นสิ่งที่ควรเลือกเป็นอันดับแรก

11. Data Relationship

- ก่อนเขียนสูตร จะต้องทำความเข้าใจข้อมูลเบื้องต้นว่ามีทั้งหมดกี่ Table แต่ละ Table มีกี่คอลัมน์ แต่ละคอลัมน์มีข้อมูลประเภทใดบ้าง และแต่ละ Table มีคอลัมน์ใดที่สัมพันธ์กัน และสัมพันธ์กันอย่างไร ดังตัวอย่างข้อมูลจำนวนนักท่องเที่ยวชาวต่างชาติที่เข้ามาในประเทศไทย ปี พ.ศ. 2553-2562 จากสำนักงานสถิติแห่งชาติ

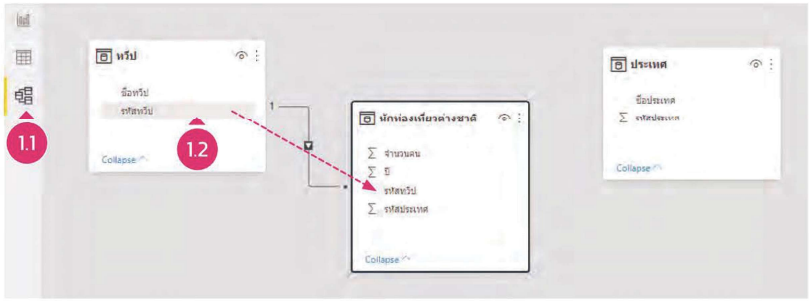


11. Data Relationship (ต่อ)

- มีข้อมูลทั้งหมด 3 ตาราง ดังนี้
  1. Table ทวีป : ใช้เก็บข้อมูลเกี่ยวกับทวีปของนักท่องเที่ยว
     - คอลัมน์ รหัสพวีป จะเป็นข้อมูลประเภท จำนวนเต็ม (Whole Number)
     - คอลัมน์ ชื่อทวีป จะเป็นข้อมูลประเภท ข้อความ (Text)
  2. Table ประเทศ : ใช้เก็บข้อมูลเกี่ยวกับประเทศของนักท่องเที่ยว
     - คอลัมน์ รหัสประเทศ จะเป็นข้อมูลประเภท จำนวนเต็ม (Whole Number)
     - คอลัมน์ ชื่อประเทศ จะเป็นข้อมูลประเภท ข้อความ (Text)
  3. Table นักท่องเที่ยวต่างชาติ : ใช้เก็บข้อมูลเกี่ยวกับจำนวนคนของนักท่องเที่ยวติ
     - คอลัมน์ รหัสทวีป จะเป็นข้อมูลประเภท จำนวนเต็ม (Whole Number)
     - คอลัมน์ รหัสประเทศ จะเป็นข้อมูลประเภท จำนวนเต็ม (Whole Number)
     - คอลัมน์ ปี จะเป็นข้อมูลประเภท ข้อความ (Text)
     - คอลัมน์ จำนวนคน จะเป็นข้อมูลประเภท จำนวนเต็ม (Whole Number)

11. Data Relationship (ต่อ)

- กรณีไม่มีการสร้างความสัมพันธ์ (Not Create Relationship)
  1. จะพบว่าในมุมมอง Model View จะไม่เห็นความสัมพันธ์ของ Table
- กรณีมีการสร้างความสัมพันธ์ (Create Relationship)
  1. จะพบว่าในมุมมอง Model View จะไม่เห็นความสัมพันธ์ของ Table
  2. มุมมอง Report View จะเห็นว่าตัวเลขในตาราง Visualize มีการเปลี่ยนแปลง โดยสามารถแยกจำนวนนักท่องเที่ยวในแต่ละทวีปได้ เนื่องจากมีการสร้างความสัมพันธ์กันแล้ว



12. Groups of DAX

- เนื่องจากการเรียนรู้เกี่ยวกับสูตร DAX นั้นก็คือ สูตรในการคำนวณ ซึ่งมีอยู่ทั้งหมดมากกว่า 250 ฟังก์ชัน แต่สามารถแบ่งออกเป็นกลุ่มต่าง ๆ ได้ 10 กลุ่ม ดังนี้

Date and time functions	Math and Trig functions
Time-intelligence functions	Other functions
Filter functions	Parent and Child functions
Information functions	Statistical functions
Logical functions	Text functions

13. 13 Functions หลักของ DAX

- https://docs.microsoft.com/th-th/dax/dax-function-reference
- https://dax.guide/functions/

1. SUM	5. CALCULATE	9. OR (||)
2. SUMX	6. KEEPFILTERS	10. IN
3. MIN	7. FILTER	11. SWITCH
4. MAX	8. AND (&&)	12. VAR
		13. Related

14. ตัวอย่างของ DAX Object Names

Object Types	Examples	Comment
Table Name	Sales	ถ้าชื่อตารางไม่มีการเว้นวรรค หรืออักขระพิเศษอื่นๆ ก็ไม่จำเป็นต้องใส่ Single Quotes
Table Name	'Canada Sales'	ถ้าชื่อตารางมีการเว้นวรรค แท็บ หรืออักขระพิเศษอื่นๆ ให้ใส่ชื่อในเครื่องหมาย Single Quotes ด้วย
Column Name ตรงตามหลักเกณฑ์	Sales[Amount]	ชื่อตารางจะอยู่หน้าชื่อคอลัมน์ และชื่อคอลัมน์จะอยู่ในเครื่องหมายวงเล็บ
Measure Name ตรงตามหลักเกณฑ์	Sales[Profit]	ชื่อตารางอยู่หน้า Measure Name และ Measure Name อยู่ในเครื่องหมายวงเล็บ โดยปกติแล้ว ต้องใช้ชื่อที่ตรงตามหลักเกณฑ์เสมอ
Column Name ไม่ตรงตามหลักเกณฑ์	[Amount]	ใช้ชื่อคอลัมน์ในเครื่องหมายวงเล็บโดยไม่ระบุชื่อตาราง กรณีนี้จะใช้เฉพาะในสูตรที่มีการหาค่าภายในคอลัมน์ของตารางเดียวกัน หรือใน Aggregation Function ที่มีการสแกนภายในตารางเดียวกันเท่านั้น
คอลัมน์ในตารางที่ตรงตามหลักเกณฑ์ กรณีมีเว้นวรรคที่ชื่อตาราง	'Canada Sales'[Qty]	หากมีการเว้นวรรคชื่อตาราง จะต้องใส่เครื่องหมาย Single Quotes ด้วย

15. DAX-SUM Function

- หลังจากสร้าง Data Modeling เรียบร้อยแล้ว การใช้ฟังก์ชัน SUM ซึ่งเป็น DAX Functions ที่จัดอยู่ในกลุ่ม Math and Trig Functions ใช้สำหรับรวมตัวเลขข้อมูลทั้งคอลัมน์ โดยเรียกว่า "ผลรวมในแนวตั้ง" ค่าผลรวมที่ส่งกลับมาจะเป็นเลขฐานสิบ



▶ การรวมข้อมูลทั้งคอลัมน์ หรือการรวมในแนวตั้ง

15. DAX-SUM Function (ต่อ)

Syntax :

SUM(<ColumnName >)

Example :

=SUM(Sales[Amt])

// หาผลรวมของตัวเลขทั้งหมดในคอลัมน์ Amt จากตาราง Sales

Parameter	Attributes	Description
ColumnName		คอลัมน์เก็บค่าตัวเลขที่หาผลรวมได้

16. ตัวดำเนินการที่ใช้ในภาษา DAX หรือ DAX Operators

- DAX เป็นภาษาที่ใช้ตัวดำเนินการเพื่อสร้างนิพจน์ (Expressions) สำหรับการเปรียบเทียบ คำนวณ ตัวเลข หรือทำงานกับสตริง เพราะฉะนั้น จึงมีตัวดำเนินการแต่ละประเภท (Types of Operators) ดังนี้
  - ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators)
  - ตัวดำเนินการเปรียบเทียบ (Comparison Operators)
  - ตัวดำเนินการต่อข้อความ (Text Concatenation Operators)
  - ตัวดำเนินการทางตรรกะ (Logical Operators)

17. ตัวอย่างตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators)

Arithmetic Operator	Meaning	Example
+ (plus sign)	Addition	3+3
– (minus sign)	Subtraction or sign	3–1–1
* (asterisk)	Multiplication	3*3
/ (forward slash)	Division	3/3
^ (caret)	Exponentiation	16^4

- นอกจากนี้ยังต้องศึกษาลำดับความสำคัญ (Precedence Order) ของตัวดำเนินการต่าง ๆ สามารถดูตัวดำเนินการประเภทอื่น ๆ ได้ที่เว็บไซต์
- Https://docs.microsoft.com/en-us/dax/dax-operator-reference

18. DAX-MIN Function

- MIN Function คือ ฟังก์ชัน DAX ที่จัดอยู่ในกลุ่ม Statistical Functions เป็นฟังก์ชันที่ใช้ในการหาจำนวนที่น้อยที่สุด โดยจะส่งคืนค่าที่น้อยที่สุดในคอลัมน์ หรือระหว่างนิพจน์สเกลาร์ 2 นิพจน์

Syntax :

MIN (< column>)

MIN(<expressionl>, <expression2>)

MIN (<ColumnNameOrScalar1> [, Scalar2])

Example :

= MIN([ResellerMargin])

//ส่งคืนค่าที่น้อยที่สุดเพียงค่าเดียวจากคอลุ้มน์ที่ถูกคำนวณคือ ResellerMargin

= MIN([TransactionDate])

//ส่งคืนค่าที่น้อยที่สุดเพียงค่าเดียวจากคอลัมน์ TransactionDate ที่มีค่าวันที่และเวลา สูตรนี้จึงส่งกลับวันที่ล่าสุด

= Min([TotalSales], [Totalpurchases])

//ได้ผลลัพธ์กลับมาค่าเดียวเป็นค่าที่น้อยที่สุดจาก 2 นิพจน์

18. DAX-MIN Function (ต่อ)

Parameter	Attributes	Description
ColumnNameOrScalar1		ColumnName : เลือกคอลัมน์ที่ต้องการหาค่าที่น้อยที่สุด Scalar1 : ใส่ค่าแรกที่ต้องการเปรียบเทียบ
Scalar2	Optional	Scalar2 : ใส่ค่าที่สองที่ต้องการเปรียบเทียบ

19. DAX-MAX Function

- MAX Function คือ ฟังก์ชัน DAX ที่จัดอยู่ในกลุ่ม Statistical Functions เช่นเดียวกับ MIN Function แต่เป็นฟังก์ชันที่ใช้หาค่าที่มากที่สุดในคอลัมน์ หรือระหว่างนิพจน์สเกลาร์ 2 นิพจน์ แต่ไม่สามารถเปรียบเทียบค่าตัวแปรชนิดตรรกะได้ (True/False) กรณีนี้ให้ใช้ MAX Function แทน ในกรณีสตริง (Strings) จะถูกเปรียบเทียบตามลำดับตัวอักษร

Syntax :

MAX (< column>)

MAX(<expression1>, <expression2>)

MAX <<ColumnNameOrScalar1> [, <Scalar2>])

Example :

= MAX(InternetSales [ExtendedAmount])

//ส่งคืนค่าที่มากที่สุดที่พบในคอลัมน์ ExtendedAmount ของตาราง InternetSales

= Max([TotalSales), (TotalPurchases))

//ส่งคืนค่าที่มากที่สุดระหว่างผลลัพธ์ของนิพจน์สองนิพจน์

19. DAX-MAX Function (ต่อ)

Parameter	Attributes	Description
ColumnNameOrScalar1		ColumnName : เลือกคอลัมน์ที่ต้องการหาค่าที่มากที่สุด Scalar1 : ใส่ค่าแรกที่ต้องการเปรียบเทียบ
expression		นิพจน์ DAX ใดๆ ที่ส่งคืนค่าเดียว
Scalar2	Optional	Scalar2 : ใส่ค่าที่สองที่ต้องการเปรียบเทียบ

20. ตัวอย่างการใช้ DAX-MAX Function

- เลือกคอลัมน์ จำนวนคน จากตาราง นักท่องเที่ยวต่างชาติ



- สามารถทราบข้อมูลในเบื้องต้นได้ว่า นักท่องเที่ยวที่มีจำนวนมากที่สุดในข้อมูลก็คือ 10,997,338 คน ซึ่งข้อมูลนี้ต้องนำไปวิเคราะห์ต่ออีกว่าเป็นทวีปใด ประเทศใด และปีใด

21. DAX-CALCULATE Function

- CALCULATE Function คือ ฟังก์ชัน DAX ที่จัดอยู่ในกลุ่ม Filter Functions เป็นฟังก์ชันสำหรับคำนวณค่านิพจน์ตามเงื่อนไขของฟิลเตอร์ที่กำหนด เป็นฟังก์ชันที่มีประโยชน์ในการคำนวณค่าค่อนข้างมาก

Syntax :

CALCULATE(<expression>[, <filter1> |, <filter2> [,...]])

Example :

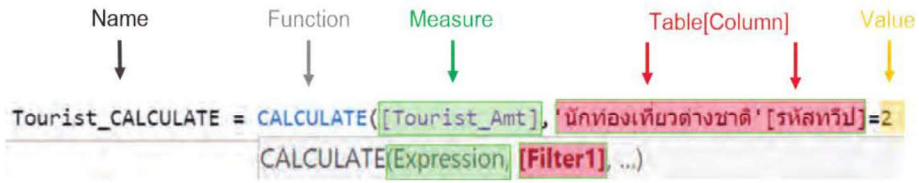
```
Blue Revenue =
CALCULATE (
SUM(Sales[Sales Amount]),
'Product'[Color] = "Blue"
)
//ต้องการประมาณรายได้จากการขายสินค้าในตาราง sales โดยเลือกเฉพาะสินค้าที่มีสีน้ำเงินเท่านั้น
```


21. DAX-CALCULATE Function (ต่อ)

Parameter	Attributes	Description
Expression		สูตรที่ต้องการคำนวณ (SUM, COUNT เป็นต้น)
filter1, filter2, ...	Optional Repeatable	ใส่ค่าที่ต้องการ Filter

22. ตัวอย่าง DAX-CALCULATE Function

- ใส่ค่ารหัสทวีปของยุโรป Value : 2



23. DAX-KEEPFILTERS Function

- FUNCTION KEEPFILTERS Function ถูกจัดอยู่ในกลุ่ม Filter Functions เป็นเหมือนกับตัวแปลงฟิลเตอร์ (Filter Modifier) ที่จะไม่ทิ้งเงื่อนไขฟิลเตอร์ที่มีอยู่เดิมในคอลัมน์หรือตาราง แต่ฟังก์ชันนี้จะเข้าไปเป็นส่วนหนึ่งของฟังก์ชัน CALCULATE และ CALCULATETABLE เพื่อแทนที่ลักษณะการทำงานมาตรฐานของฟังก์ชันเหล่านั้น

23. DAX-KEEPFILTERS Function (ต่อ)

```
Syntax :
KEEPFILTERS(<expression>)

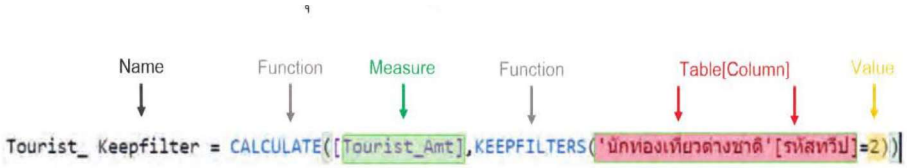
Example:
    1   Always Red Sales :=
    2   CALCULATE(
    3       [Sales Amount],
    4       'Product'[Color] = "Red"
    5   )
    6
    7   Only Red Sales :=
    8   CALCULATE (
    9       [Sales Amount],
    10      KEEPFILTERS( 'Product'[Color] = "Red" )
    11  )

//Always Red Sales รับค่ายอดขายรวมของสินค้าสีแดงจากตัวกรองหลัก CALCULATE
//Only Red Sales รับค่ายอดขายของสินค้าเฉพาะสีแดงจากสีที่มีให้เลือก โดยซ้อน Filter Modifier
ไว้ในตัวกรองเดิมของคอลัมน์ Color หากไม่ตรงตามเงื่อนไขล่าสุดจะได้ผลลัพธ์เป็นค่าว่าง (Blank)
```

Parameter	Attributes	Description
Expression		Expression หรือ Filter ที่ใช้ CALCULATE หรือ CALCULATETABLE function

24. ตังอย่าง DAX-KEEPFILTERS Function

- ใส่ค่ารหัสทวีปของยุโรป Value : 2



25. DAX-FILTER Function

- FILTER Function ถูกจัดอยู่ในกลุ่ม Filter Functions (ชื่อเดียวกับชื่อกลุ่ม) เป็นฟังชั่นที่ให้ผลลัพธ์เป็นตารางหรือนิพจน์ที่มีเฉพาะแถวที่เข้าเงื่อนไขของฟิลเตอร์เท่านั้น โดยฟังก์ชัน KEEPFILTERS ใช้ฟิลเตอร์ได้เพียงเงื่อนไขเดียวเท่านั้น ฉะนั้นถ้าต้องการฟิลเตอร์มากกว่า 1 เงื่อนไขขึ้นไป ก็ต้องเปลี่ยนมาใช้งานฟังก์ชัน FILTER ที่ รองรับเงื่อนไขได้มากกว่า

25. DAX-FILTER Function (ต่อ)

Syntax :

FILTER(<<table>,<filter>)

FILTER (<Table>), <FilterExpression>)

Example :

FILTER('InternetSales_USD', RELATED('SalesTerritory' [SalesTerritoryCoun-try])<>"United States")

//นิพจน์ (Expression) นี่คือ การสร้างรายงานยอดขายทางอินเทอร์เน็ตนอกอเบริกา โดยใช้ Measure กรองยอดขายที่เกิดขึ้นในอเบริกาออก แล้วแบ่งข้อมูลตามปีปฏิทิน และแบ่งกลุ่มสิบสินค้า โดยสร้าง Measure กรองตาราง Internet sales USD ในส่วนของ Sales Territory หรือพื้นที่ขาย

SUMX(FILTER('InternetSales_USD',

RELATED(SalesTerritory'[SalesTerritoryCountry])<>"United State ") InternetSales_USD'

[SalesAmount_USD])

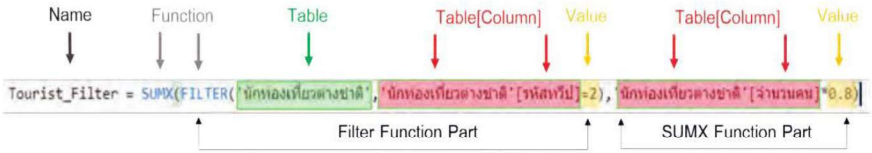
//จากนั้นให้ใช้ตารางที่กรองเอายอดขายในอเบริกาออก แล้วมาหาผลรวมทั้งหมดด้วยฟังก์ชัน SUMX

//รายละเอียดเพิ่มเติม : https//docs microsoft com/en-us/dax/flter-function-dax

25. DAX-FILTER Function (ต่อ)

Parameter	Attributes	Description
Table		ตารางที่จะใส่ฟิลเตอร์ หรือนิพจน์ที่ทำให้เกิดตารางได้ด้วย
FilterExpression		นิพจน์บูลีน (True/False) ที่ถูกฟิลเตอร์ในแต่ละแถวของตาราง เช่น [Amount] > 0 หรือ [Region] = "France"

26. ตัวอย่าง DAX-FILTER Function



27. DAX-AND Function

- AND Function ถูกจัดอยู่ในกลุ่ม Logical Functions เป็นฟังก์ชั้นที่จะตรวจสอบเงื่อนไข ถ้าอาร์กิวเมนต์ทั้งคู่นั้นเป็น TRUE ก็จะให้ผลลัพธ์เป็น TRUE แต่ถ้าไม่ก็จะแสดงค่า FALSE โดยจะให้ค่า TRUE ก็ต่อเมื่อเข้าทุกเงื่อนไข และจะให้ค่า FALSE เมื่อเงื่อนไขใดเงื่อนไขหนึ่งไม่ตรง ดังนั้น ผลลัพธ์จะเป็น TRUE หรือ FALSE จึงขึ้นอยู่กับการรวมของค่าที่นำมาทดสอบโดยฟังก์ชัน AND ใน DAX จะมีเพียง 2 อาร์กิวเมนต์เท่านั้น หากต้องการดำเนินการ AND กับหลาย ๆ นิพจน์ โอเปอร์เรเตอร์ And (&&) เพื่อรวมนิพจน์ทั้งหมดจะง่ายกว่า โดยโอเปอเรเตอร์ลอจิก AND (88) จะให้เป็น TRUE ถ้าอาร์กิวเมนต์ทั้งคู่เป็น TRUE แต่ถ้ามีค่าหนึ่งค่าใดเป็น FALSE ผลลัพท์ก็จะเป็น FALSE

27. DAX-AND Function (ต่อ)

Syntax :

AND(<Logicall>,<Logical2>)

Example :

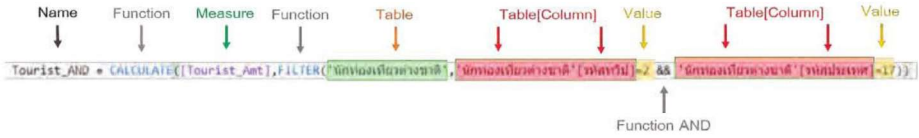
```
--AND Function
IF(AND(10>9,-10<-1), "All TRUE", "One or more FALSE"
//ฟังก์ชัน AND จะเป็นจริงก็ต่อเมื่อผลลัพธ์ที่ได้ "จริงทั้งหมด" ฟังก์ชัน AND จะเป็นเท็จ ถ้ามีค่าใดค่าหนึ่งเป็นเท็จ
--AND (&&) Operator
[Measure B] = 30 && [measure A] > 10 && [measure )
AND (
[measure B] = 30,
AND (
[measure A] > 10,
[measure A] < 20
)
)
```

27. DAX-AND Function (ต่อ)

Parameter	Attributes	Description
Logical1		ค่าเชิงตรรกะที่ต้องการทดสอบ
Logical2		

28. ตัวอย่าง DAX-AND Function

1. กำหนดค่าต่างๆ ในสูตร DAX ดังนี้ แล้วกดปุ่ม <Enter> เมื่อจบขั้นตอนกำหนดค่าในสูตรแล้ว
   1.1 ใส่ชื่อ Tourist_AND แทน Measure
   1.2 เลือกฟังก์ชัน CALCULATE, FILTER, AND
   1.3 ที่ฟังก์ชัน FILTER ใส่เงื่อนไขที่ต้องการหา ดังรูป



2. จะปรากฏคอลัมน์ Tourist_AND ภายในตาราง นักท่องเที่ยวต่างชาติ โดยจะมีสัญลักษณ์เครื่องเลขกำกับ เพื่อแสดงว่าถูกสร้างโดย Measure แต่ยังไม่สามารถรู้ได้ว่า การคำนวณนี้มีค่าเท่าไร จนกว่าจะมีการ Visualize ในมุมมอง Report

29. DAX-OR Function

- OR Function ถูกจัดอยู่ในกลุ่ม Logical Functions ใช้ตรวจสอบว่า ถ้าหนึ่งในอาร์กิวเมนต์เป็นก็จะได้ผลลัพธ์หรือเอาต์พุตเป็น "จริง หรือ TRUE" แต่จะให้ผลลัพธ์เป็น FALSE ก็ต่อเมื่ออาร์กิวสองเป็น "เท็จ หรือ FALSE" พูดง่าย ๆ ก็คือฟังก์ชันนี้จะตรวจสอบเงื่อนไข ผลลัพธ์จะเป็น TRUE ก็ต่อเมื่อเงื่อนไขใดเงื่อนไขหนึ่งเป็นจริง แต่จะเป็น FALSE เพียงกรณีเดียวคือ ตรวจสอบแล้วพบว่า ทุกเงื่อนไขเป็นเท็จ โดยฟังก์ชัน OR ใน DAX จะมีเพียง 2 อาร์กิวเมนต์เท่านั้น หากต้องการดำเนินการ AND กับหลาย ๆ นิพจน์ให้ใช้โอเปอเรเตอร์ OR (II) เพื่อรวมนิพจน์ทั้งหมดจะง่ายกว่า โดยโอเปอเรเตอร์ลอจิก OR (II) จะให้ผลลัพธ์เป็น TRUE ถ้าอาร์กิวเมนต์ค่าหนึ่งค่าใดเป็น TRUE แต่ถ้าอาร์กิวเมนต์ทั้งสองเป็น FALSE ผลลัพธ์ก็จะเป็น FALSE

29. DAX-OR Function (ต่อ)

Syntax :

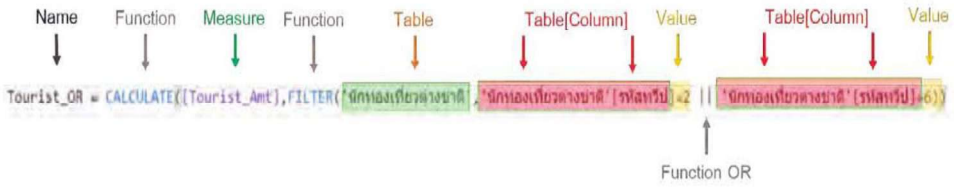
OR(<logical1>,<logical2>)

Example :

```
--OR Function
[measure A] > 10,
OR (
[measure B] > 20,
[measure C] = 30
)
)
--Or ( | | )Operator
[Measure A] > 10 || [measure B] > 20 || [measure C]=30
```

30. ตัวอย่าง DAX-OR Function

- ที่ฟังก์ชัน FILTER ใส่เงื่อนไขที่ต้องการหา ดังรูป



31. DAX-IN Operator

- IN Operator ตัวนี้ไม่ใช่ฟังก์ชัน แต่เป็นโอเปอเรเตอร์ในกลุ่ม "Logical Operators“ ซึ่งเป็นกลุ่ม เดียวกับโอเปอเรเตอร์ลอจิก AND (&& : Double Ampersand), โอเปอเรเตอร์ลอจิก OR (II : Double Pipe Symbol) ซึ่งอาจเรียกกลุ่มนี้ว่า "Text Operator" ก็ได้ เราใช้ IN Operator เพื่อสร้างเงื่อนไขแบ ลอจิก OR ที่นำค่า Logical1, Logical2 แต่ละแถวมาเปรียบเทียบกัน ผลลัพธ์จะเป็น TRUE ก็ต่อเมื่อเงื่อนไขใด เงื่อนไขหนึ่งเป็นจริง แต่จะเป็น FALSE เพียงกรณีเดียวคือ ตรวจสอบแล้วพบว่า ทุกเงื่อนไขเป็นเท็จ โดยใน Syntax จะใช้วงเล็บปีกกา (Curly Braces) {} ซึ่งช่วยให้เขียนสูตร DAX ได้สิ้นกระชับขึ้น

31. DAX-IN Operator (ต่อ)

```
Syntax :
OR(<Logical1>,<Logical2>)

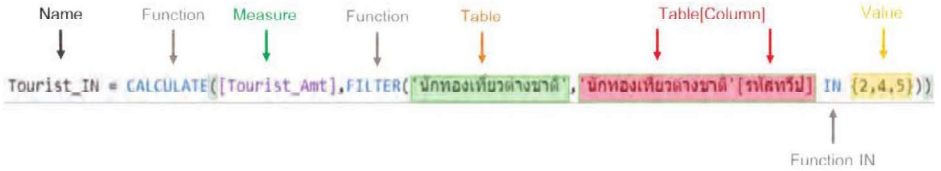
Example:
'Product'[Color] IN { "Red", "Blue", "Black" }
```

```
    -- OR Syntax
1   RedOrBlack Sales OR :=
2   CALCULATE (
3       [Sales Amount],
4       Products[Color] = "Red" || Products[Color] = "Black"
5   )
```

```
    --IN Syntax:
1   RedOrBlack Sales IN :=
2   CALCULATE (
3       [Sales Amount],
4       Products[Color] IN { "Red", "Black" }
5   )
```


32. ตัวอย่าง DAX-IN Operator

- ที่ฟังก์ชัน FILTER ใส่เงื่อนไขที่ต้องการหาในวงเล็บปีกกา



- จะเห็นว่าเราสามารถค้นหาข้อมูลมากกว่า 1 เงื่อนไขได้ โดยใช้ฟังก์ชัน IN ซึ่งเงื่อนไขที่ต้องการหานั้นก็คือ นักท่องเที่ยวทวีปยุโรป นักท่องเที่ยวทวีปเอเชียใต้ หรือนักท่องเที่ยวทวีปโอเชียเนีย ถ้ามีข้อมูลใดข้อมูลหนึ่งก็จะแสดงค่าจริง (TRUE) และจะทราบว่า ข้อมูลนักท่องเที่ยวทวีปยุโรป นักท่องเที่ยวทวีปเอเชียใต้ หรือนักท่องเที่ยวทวีปโอเชียเนีย ที่มาเที่ยวประเทศไทยมีจำนวนเท่าไร

33. DAX-SWITCH Function

- SWITCH Function ถูกจัดอยู่ในกลุ่ม Logical Functions หาค่านิพจน์โดยเทียบกับค่าในลิสต์ และให้ผลลัพธ์กลับมาเพียงค่าเดียว จะเป็นฟังก์ชันตรวจสอบเงื่อนไข ซึ่งจะให้ค่าที่เราต้องการ ขึ้นอยู่กับเงื่อนไขที่เราใส่เข้าไป หรือเปรียบการทำงานของฟังก์ชัน SWITCH ก็จะคล้ายๆ กับคำสั่ง IF ควบคุมหลายเงื่อนไข (Multiple IF)

33. DAX-SWITCH Function (ต่อ)

Syntax :

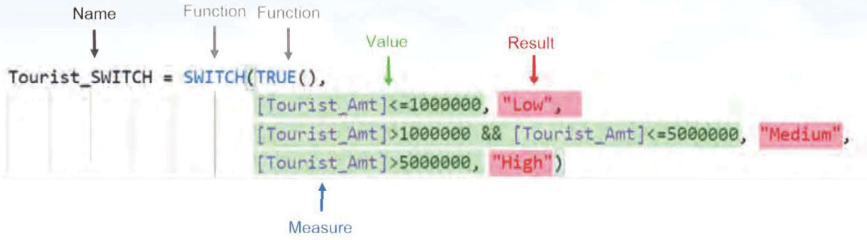
SWITCH(<Expression>,<Value>,<Result>[,<Value>, <Result>]....[, <Else>])

Example :

= SWITCH([Month), 1, "January", 2, "February", 3, "March", 4, "April", 5, "May"
,6, "June", 7, "July", 8, "August", 9, "September"
, 10, "October", 11, "November", 12, "December"
, "Unknown month number")
//สร้างคอลัมน์เปลี่ยนชื่อเดือนตามนับเบอร์ที่คำนวณได้

Parameter	Attributes	Description
Expression		สูตรหรือเงื่อนไขที่ต้องการ
Value	Repeatable	เงื่อนไขที่ต้องการค้นหา
Result	Repeatable	ผลลัพธ์ที่ต้องการแสดง เมื่อเข้าเงื่อนไข
Else	Optional	ผลลัพธ์ที่ต้องการ ถ้าไม่เข้าเงื่อนไขใดๆ เลย

34. ตัวอย่าง DAX-SWITCH Function



- จะเห็นว่าเราสามารถค้นหาข้อมูลมากกว่า 1 เงื่อนไข (Multible IF) โดยใช้ฟังก์ชัน SWITCH ซึ่งจะทำให้เราวิเคราะห์ข้อมูลได้อีกมุมมองหนึ่ง โดยการแบ่งกลุ่มข้อมูลประเทศที่เข้าเงื่อนไข Low, Medium, High

35. DAX-VAR Function

- VAR Function จัดอยู่ใน DAX Statements ซึ่งกลุ่มนี้ประกอบด้วย DEFINE, EVALUATE, ORDER BY โดยฟังก์ชัน VAR จะย่อมาจาก Variable จะเป็นฟังก์ชันที่ใช้เก็บค่าซึ่งจะคล้าย ๆ กับกล่องเก็บสูตร

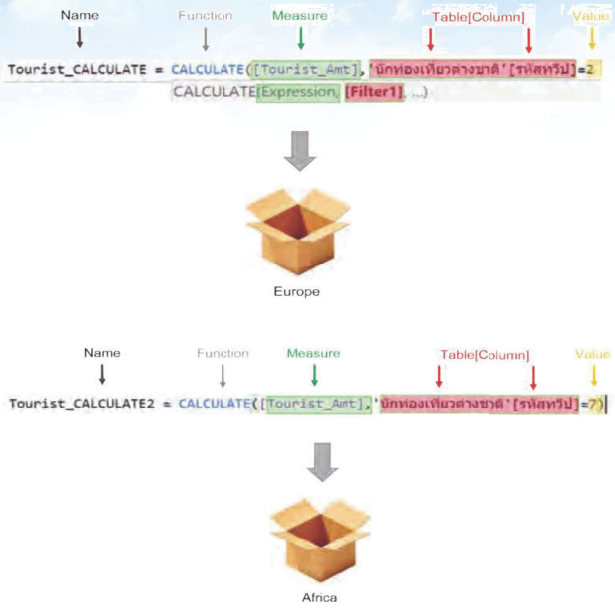
Syntax :

VAR <Name> = <Expression>

35. DAX-VAR Function (ต่อ)

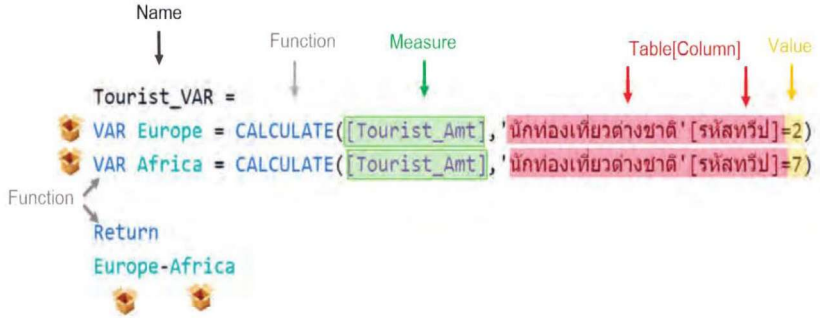
Term	Definition
Name	ชื่อของตัวแปร (ชื่อเฉพาะ) ใช้ตัวคั่นปนกับชื่อไม่ได้ เช่น 'varName' หรือ [varName] จะทำให้เออเรอร์ ชุดอักขระที่ใช้ได้ : a-z, A-Z, 0-9 แต่เลข 0-9 จะต้องไม่ใช่อักษรตัวแรก สามารถใช้ __ (Double Underscore) เป็นคำนำหน้าชื่อเฉพาะได้ ไม่ให้ใช้อักขระพิเศษต่างๆ (special characters) ไม่ให้ใช้คีย์เวิร์ดที่สงวนไว้ (Reserved keywords) ไม่ให้ใช้ชื่อซ้ำกับชื่อตาราง ไม่ให้ใช้ช่องว่าง
Expression	นิพจน์ DAX ที่ให้ผลลัพธ์เป็นค่าสเกลาร์หรือตาราง

36. ตัวอย่าง DAX-VAR Function



36. ตัวอย่าง DAX-VAR Function (ต่อ)

- กำหนดค่าในสูตรฟังก์ชัน VAR เพื่อเก็บค่าผ่านตัวแปรชื่อ Europe และ Africa แล้วคืนผลต่างระหว่าง Europe ลบด้วย Africa กลับมา

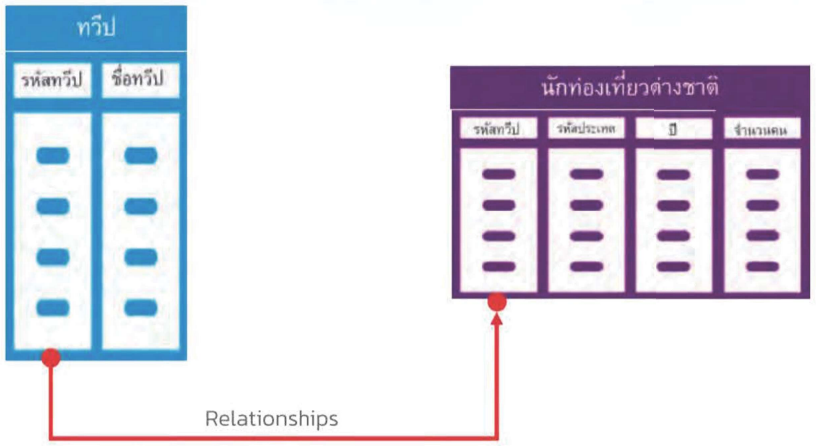


37. DAX-RELATED Function

- RELATED Function จัดอยู่ในกลุ่ม Relationship Functions เป็นฟังก์ชั้นในการค้นหาข้อมูลระหว่างตาราง ซึ่งตารางที่จะทำการค้นหานั้นจะต้องมีความสัมพันธ์กัน จึงจะสามารถใช้ฟังก์ชันนี้ได้ เช่น รหัสทวีป ในตารางนักท่องเที่ยวต่างชาติ มีความสัมพันธ์กับคอลัมน์ รหัสทวีป ในตารางทวีป

37. DAX-RELATED Function (ต่อ)

- แสดงความสัมพันธ์ของคอลัมน์รหัสทวีปของทั้ง 2 ตาราง



37. DAX-RELATED Function (ต่อ)

Syntax :

RELATED(<ColumnName>)

Example :

FILTER('InternetSalesUSD',RELATED('SalesTerritory'(SalesTerritoryCountry])<>"United States")

Parameter	Attributes	Description
ColumnName		คอลัมน์ที่ต้องการค้นหา และสามารถเชื่อมความสัมพันธ์กันระหว่างตารางได้

38. ตัวอย่าง DAX-RELATED Function

- จะเห็นว่าฟังก์ชั้น RELATED ต้องการให้ใส่ ColumnName ให้เลือกคอลัมน์ ชื่อทวีปจากตาราง ทวีป

